

value_name, x
ident
integer_literal
label, l, k field label
index, i, j, n, m

<i>typeconstr</i>	$::=$ unit bool int		
<i>typvar</i>	$::=$ <i>'ident</i>		
<i>typeexpr, t</i>	$::=$ <i>typvar</i> <i>typeexpr</i> $\rightarrow$ <i>typeexpr'</i> <i>typeconstr</i> $\{l_1 : \textit{typeexpr}_1, \dots, l_n : \textit{typeexpr}_n\}$ (<i>typeexpr</i>)	M	
<i>typescheme</i>	$::=$ (<i>typvar</i> ₁ , .., <i>typvar</i> _n) <i>typeexpr</i> generalise (Γ, t)	bind <i>typvar</i> ₁ .. <i>typvar</i> _n in <i>typeexpr</i> M	
<i>constant</i>	$::=$ <i>integer_literal</i> false true () (&&) not		
<i>expr, e</i>	$::=$ <i>value_name</i> <i>constant</i> <i>expr expr'</i> function <i>value_name</i> $\rightarrow$ <i>expr</i> let <i>p</i> = <i>expr</i> in <i>expr'</i> $\{l_1 = e_1, \dots, l_n = e_n\}$ <i>e.l</i> (<i>expr</i>) $\{v/x\}e$ σe	bind <i>value_name</i> in <i>expr</i> bind b(<i>p</i>) in <i>expr'</i> M M M	 projection
<i>p</i>	$::=$ <i>x</i> $\{l_1 = p_1, \dots, l_n = p_n\}$	b = <i>x</i> b = b(<i>p</i> ₁ .. <i>p</i> _n)	pattern variable pattern
<i>Value, v</i>	$::=$ <i>constant</i> function <i>value_name</i> $\rightarrow$ <i>expr</i> $\{l_1 = v_1, \dots, l_n = v_n\}$		
Γ, Δ	$::=$		

		empty	
		$\Gamma, value_name : \text{typscheme}$	
		$\Gamma_1, \dots, \Gamma_n$	M
		generalise (Γ, Δ)	M
σ	::=		multiple term substitution
		$[x \mapsto e]$	
		$\sigma_1, \dots, \sigma_n$	
<i>formula</i>	::=		
		<i>judgement</i>	
		not (<i>formula</i>)	
		(<i>formula</i>)	
		$\text{typscheme} > t$	
		$\text{typscheme} = \text{typscheme}'$	
		$value_name = value_name'$	
		$formula_1 \dots formula_n$	
		$\forall i \in 1..m. formula$	
		$\exists i \in 1..m. formula$	
		$formula / \setminus formula'$	
		$l = l'$	
<i>terminals</i>	::=		
		$\rightarrow$	
		function	
		$\vdash$	
		$\longrightarrow$	
		{	
		}	
		$\Rightarrow$	
		$\mapsto$	
<i>Jtype</i>	::=		
		$value_name : \text{typscheme}$ in Γ	
		$\Gamma \vdash constant : t$	
		$\Gamma \vdash e : t$	
		$\vdash p : t \Rightarrow \Delta$	
<i>Jop</i>	::=		
		$e \longrightarrow e'$	
		match (p, v) = σ	
<i>judgement</i>	::=		
		<i>Jtype</i>	
		<i>Jop</i>	
<i>user_syntax</i>	::=		
		<i>value_name</i>	
		<i>ident</i>	

$integer_literal$
 $label$
 $index$
 $typeconstr$
 $typvar$
 $typeexpr$
 $typescheme$
 $constant$
 $expr$
 p
 $Value$
 Γ
 σ
 $formula$
 $terminals$

$value_name : typescheme \text{ in } \Gamma$

$$\begin{array}{c}
\frac{}{value_name : typescheme \text{ in } \Gamma, value_name : typescheme} \quad VTS_{IN_VN1} \\
\frac{value_name : typescheme \text{ in } \Gamma \quad \mathbf{not} (value_name = value_name')}{value_name : typescheme \text{ in } \Gamma, value_name' : typescheme'} \quad VTS_{IN_VN2}
\end{array}$$

$\Gamma \vdash constant : t$

$$\begin{array}{c}
\frac{}{\Gamma \vdash integer_literal : \mathbf{int}} \quad \text{CONSTANT_INT} \\
\frac{}{\Gamma \vdash \mathbf{false} : \mathbf{bool}} \quad \text{CONSTANT_FALSE} \\
\frac{}{\Gamma \vdash \mathbf{true} : \mathbf{bool}} \quad \text{CONSTANT_TRUE} \\
\frac{}{\Gamma \vdash () : \mathbf{unit}} \quad \text{CONSTANT_UNIT} \\
\frac{}{\Gamma \vdash (\&\&) : \mathbf{bool} \rightarrow (\mathbf{bool} \rightarrow \mathbf{bool})} \quad \text{CONSTANT_AND} \\
\frac{}{\Gamma \vdash \mathbf{not} : \mathbf{bool} \rightarrow \mathbf{bool}} \quad \text{CONSTANT_NOT}
\end{array}$$

$\Gamma \vdash e : t$

$$\begin{array}{c}
\frac{x : typescheme \text{ in } \Gamma \quad typescheme > t}{\Gamma \vdash x : t} \quad \text{GET_VALUE_NAME} \\
\frac{\Gamma \vdash constant : t}{\Gamma \vdash constant : t} \quad \text{GET_CONSTANT} \\
\frac{\Gamma \vdash e : t_1 \rightarrow t_2 \quad \Gamma \vdash e' : t_1}{\Gamma \vdash e e' : t_2} \quad \text{GET_APPLY} \\
\frac{\Gamma, x_1 : () t_1 \vdash e : t}{\Gamma \vdash \mathbf{function} x_1 \rightarrow e : t_1 \rightarrow t} \quad \text{GET_LAMBDA}
\end{array}$$

$$\begin{array}{c}
\frac{\Gamma \vdash e_1 : t_1 \quad \vdash p : t_1 \Rightarrow \Delta \quad \Gamma, \mathbf{generalise}(\Gamma, \Delta) \vdash e_2 : t_2}{\Gamma \vdash \mathbf{let} \ p = e_1 \ \mathbf{in} \ e_2 : t_2} \quad \text{GET_LET} \\
\\
\frac{\Gamma \vdash e_1 : t_1 \dots \Gamma \vdash e_n : t_n}{\Gamma \vdash \{l_1 = e_1, \dots, l_n = e_n\} : \{l_1 : t_1, \dots, l_n : t_n\}} \quad \text{GET_RCD} \\
\\
\frac{\Gamma \vdash e : \{l_1 : t_1, \dots, l_n : t_n\}}{\Gamma \vdash e.l_j : t_j} \quad \text{GET_PROJ}
\end{array}$$

$$\boxed{\vdash p : t \Rightarrow \Delta}$$

$$\begin{array}{c}
\frac{}{\vdash x : t \Rightarrow \mathbf{empty}, x : ()t} \quad \text{P_VAR} \\
\\
\frac{\vdash p_1 : t_1 \Rightarrow \Delta_1 \dots \vdash p_n : t_n \Rightarrow \Delta_n}{\vdash \{l_1 = p_1, \dots, l_n = p_n\} : \{l_1 : t_1, \dots, l_n : t_n\} \Rightarrow \Delta_1, \dots, \Delta_n} \quad \text{P_RCD}
\end{array}$$

$$\boxed{e \longrightarrow e'}$$

$$\begin{array}{c}
\frac{}{(\mathbf{function} \ x \rightarrow e) \ v \longrightarrow \{v/x\}e} \quad \text{JO_RED_APP} \\
\\
\frac{\mathbf{match}(p, v_1) = \sigma}{\mathbf{let} \ p = v_1 \ \mathbf{in} \ e_2 \longrightarrow \sigma \ e_2} \quad \text{JO_RED_LETV} \\
\\
\frac{}{\{l'_1 = v_1, \dots, l'_n = v_n\}.l'_j \longrightarrow v_j} \quad \text{JO_RED_PROJRCD} \\
\\
\frac{e \longrightarrow e'}{e \ e_1 \longrightarrow e' \ e_1} \quad \text{JO_RED_CONTEXT_APP1} \\
\\
\frac{e \longrightarrow e'}{v \ e \longrightarrow v \ e'} \quad \text{JO_RED_CONTEXT_APP2} \\
\\
\frac{e \longrightarrow e'}{\mathbf{let} \ p = e \ \mathbf{in} \ e_1 \longrightarrow \mathbf{let} \ p = e' \ \mathbf{in} \ e_1} \quad \text{JO_RED_CONTEXT_LET} \\
\\
\frac{}{\mathbf{not} \ \mathbf{true} \longrightarrow \mathbf{false}} \quad \text{JO_RED_NOT_1} \\
\\
\frac{}{\mathbf{not} \ \mathbf{false} \longrightarrow \mathbf{true}} \quad \text{JO_RED_NOT_2} \\
\\
\frac{}{((\&\&) \ \mathbf{true}) \ e \longrightarrow e} \quad \text{JO_RED_AND_1} \\
\\
\frac{}{((\&\&) \ \mathbf{false}) \ e \longrightarrow \mathbf{false}} \quad \text{JO_RED_AND_2}
\end{array}$$

$$\boxed{\mathbf{match}(p, v) = \sigma}$$

$$\begin{array}{c}
\frac{}{\mathbf{match}(x, v) = [x \mapsto v]} \quad \text{JO_M_VAR} \\
\\
\frac{\textcolor{red}{\langle\langle \text{multiple parses} \rangle\rangle}}{\mathbf{match}(\{l_1 = p_1, \dots, l_m = p_m\}, \{k_1 = v_1, \dots, k_n = v_n\}) = \sigma_1, \dots, \sigma_m} \quad \text{JO_M_RCD}
\end{array}$$

Definition rules: 28 good 1 bad
 Definition rule clauses: 47 good 1 bad